Grounded Statute Retrieval

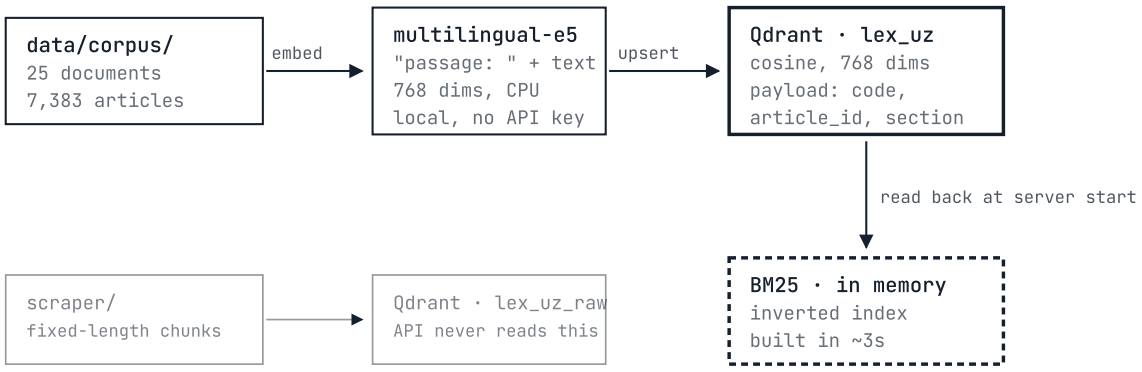
How the lex.uz RAG API finds one article among 7,383 and proves the answer came from it.

The service answers questions about Uzbek legislation. The architectural problem is not answering — a language model does that readily — but making the answer **checkable**: every claim has to trace to a specific *modda* (article), and a claim that cannot be traced must not ship.

That requirement shapes all three stages below. Retrieval is built so the right article is actually in the model's hands; generation is constrained to quoting what it was given; and a verification gate re-checks every quote against its source before the response leaves the process.

Stage 1 — Building the index

This runs once. The corpus is 25 documents (the Constitution, 20 codes, 4 laws) pinned at the `2026-07` snapshot, stored as one JSON object per article. Chunking is **by article, never by character count**, which is what makes `citations[].article` exact rather than inferred.



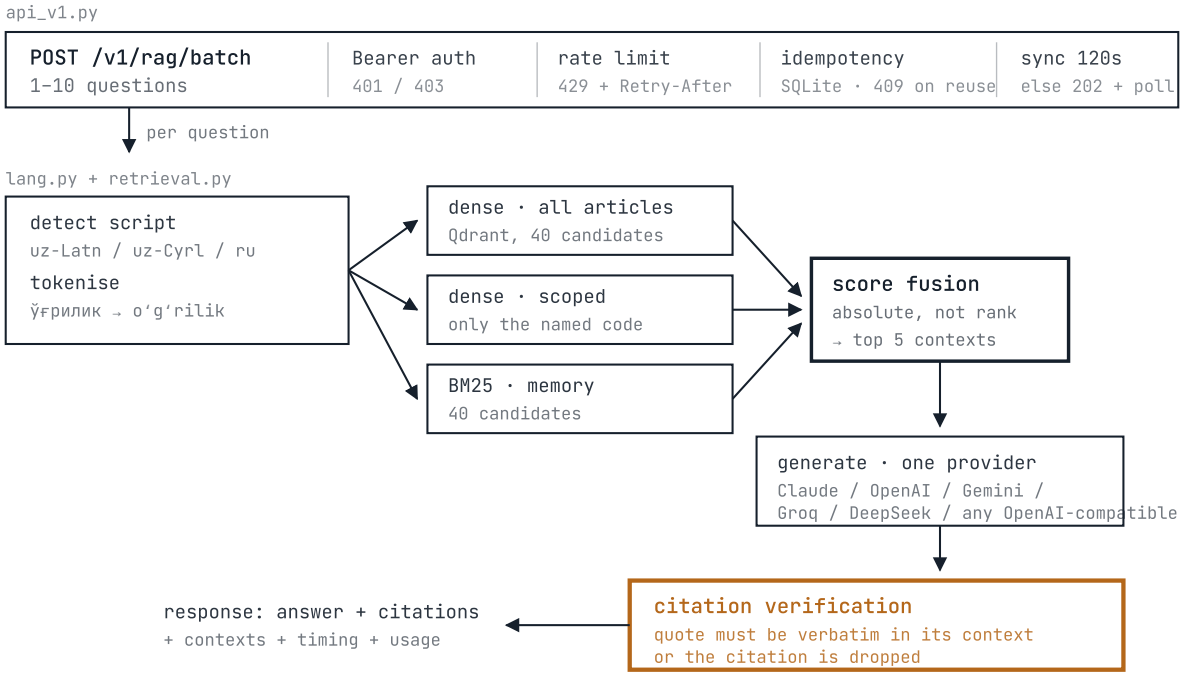
Separate collection on purpose: these chunks carry no article_id, so no valid citation can be built from one.

One article is one chunk, and the article payload feeds both retrievers — the vector store and, at startup, the lexical index. The faint path is the daily scraper, deliberately writing to a different

collection so it can never reach the index that answers questions.

Stage 2 — Answering a question

Two retrievers run on every question because each fails where the other works. Dense search matches meaning: ask *“ishdan bo‘shatishsa nima qilish kerak”* and it finds articles about *mehnat shartnomasini bekor qilish* — different words, same subject. Lexical search matches the decisive rare term: ask about *o‘g‘irlik* and BM25 lands on the article titled *O‘g‘rilik* outright.



Every request crosses four gates before any retrieval happens, then each question fans out to three candidate sources and converges on a fused top five. The last box is the one that makes the response trustworthy rather than merely plausible.

WHY THE SCORES ARE FUSED, NOT THE RANKS

Reciprocal rank fusion was the first implementation and it was wrong for this corpus. Legal articles share vocabulary heavily, so e5 similarities compress: a question’s 40 candidates typically span **0.79 to 0.81**. Dense *rank* within that band is close to a coin flip. BM25 margins, meanwhile, are wide and meaningful — an exact article-title match scores **12.5** where the next article scores **7.9**.

Rank fusion discards magnitude, which is exactly the signal that

discriminates here. The correct article for *o‘g‘irlik* sat at rank 5. Fusing absolute scores put it at rank 1:

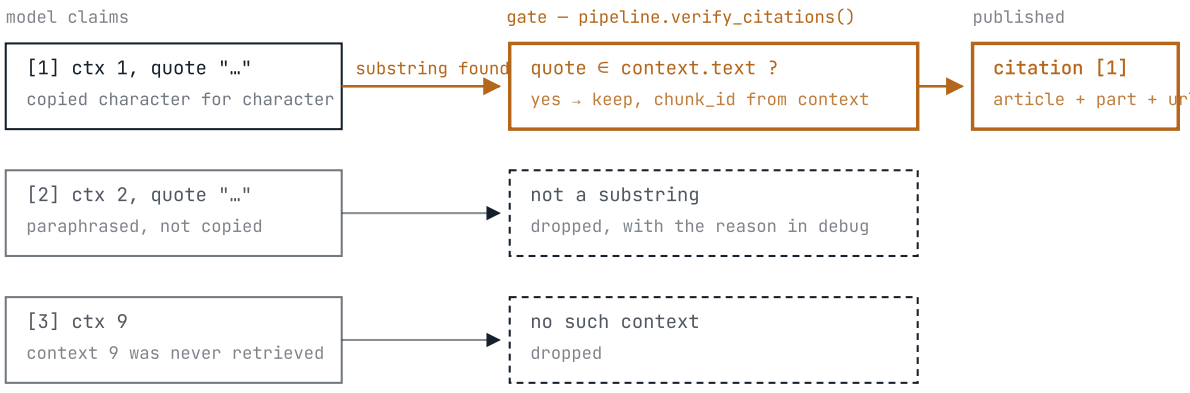
```
fused = 1.0 · dense + 0.8 · lexical + 0.3 · code_bonus

dense   = clamp((cosine - 0.75) / 0.20, 0, 1)    absolute, not pool-relative
lexical = bm25 / (bm25 + 6)                      saturating
code    = 0.3 if the question named this code    flat, so it cancels within a code
```

Each term is deliberate. `dense` maps an *absolute* cosine band, so a pool of uniformly mediocre matches cannot promote its best member to 1.0. `lexical` saturates, so a strong exact match counts for much more than incidental word overlap. The code bonus is *flat*: an earlier version added it as a second dense term, which silently doubled dense’s weight whenever a code was named and let a generic article (*Jinoyatning tushunchasi*, cosine 0.845) outscore the article whose title is the question’s decisive word.

Stage 3 — The verification gate

The model is asked for structured JSON rather than prose: an answer with bracketed markers, and for each marker the context number it came from plus the passage it copied. “Asked for” is not “guaranteed”, so every claim is re-checked on the way out.



If no citation survives, the answer is replaced by no_answer rather than published ungrounded. Markers in the answer text are renumbered onto the citations that remain.

The gate is the reason a fabricated source is structurally impossible rather than merely discouraged: `chunk_id` is read off the retrieved context, never taken from the model, so a reference to something the system never saw has nowhere to come from.

> WHAT THIS BUYS
>
> Measured over 100 questions through the HTTP endpoint: faithfulness **1.000**, citation accuracy **0.966**, fabricated sources **0**. A weaker model loses citations here instead of inventing them — which is the behaviour you want from a legal tool.

Where state lives

STORE	HOLDS	SIZE	WHY THERE
`Qdrant lex_uz`	768-dim vectors + article payloads	7,383	Payload filtering and a full scroll were both needed
`process memory`	BM25 inverted index	7,383	Corpus is small; a separate search service would earn nothing
`SQLite`	batch results, 30-day retention	—	Idempotent replay, 409 detection, async batch state
`Qdrant lex_uz_raw`	scraped documents, fixed-length chunks	60	Isolated: no article structure, so no valid citation

The embedding model is `multilingual-e5-base`, running locally on CPU with no API key and no per-query cost. Indexing and querying must use the same model and the same `passage:` / `query:` prefixes or the vectors stop being comparable, so it has exactly one home that both sides import.

Code map

`backend/retrieval.py`	Both retrievers, transliteration, code detection, score fusion
`backend/generate.py`	Prompt, JSON schema, five provider paths, parameter adaptation
`backend/pipeline.py`	Per-question orchestration and the verification gate
`backend/api_v1.py`	The five endpoints, auth, rate limit, sync/async
`backend/embedding.py`	The shared model — indexing and querying must match
`backend/store.py`	SQLite: idempotency, conflicts, async state, retention
`backend/config.py`	Settings, and provider resolution from whichever key is present
`backend/lang.py`	Script detection and apostrophe normalisation
`ingest/ingest_corpus.py`	Corpus → Qdrant `lex_uz`. Run once

`scripts/verify.py`	One command: environment, index, retrieval, 69 tests, provider
`scripts/run_eval.py`	The spec's scenario: 10 sequential batches, retries, async polling
`scripts/score_eval.py`	Hit@5, MRR, faithfulness, latency, error share
`tests/test_api_v1.py`	The specification's checklist, line by line

THE PROVIDER LAYER

Only three code paths exist: the two native APIs (Anthropic, Gemini) and the OpenAI chat-completions protocol that everything else speaks. Adding a provider is a table entry, not code. With `RAG_LLM_BACKEND=auto` the first provider whose key is present wins, so dropping any one key into `.env` is the whole setup.

Which optional parameters an endpoint accepts varies by *model*, not just by provider — some reject `temperature` and want `max_completion_tokens` instead of `max_tokens`. The request therefore starts with everything useful and drops or renames whatever the endpoint objects to, recording each adaptation in `debug`.

Retrieval spot-check across uz-Latn, uz-Cyrl and ru: 5 of 5 correct articles at rank 1. Hit@5 (document) 0.978 over 100 questions. Latency p50 0.23s excluding generation.